KNN Classification model

In [209…
```python
import pandas as pd
import numpy as np
```

Dataset

In [212…
```python
data={
    "Study_hours":[1,2,2,3,3,4,5,6,7,8],
    "attendance":[50,55,60,65,70,72,80,90,95,97],
    "result":['Fail','Fail','Fail','Fail','Fail','Pass','Pass','Pass','Pass','Pass']
}
```

In [214…
```python
data
```

Out[214…
```
{'Study_hours': [1, 2, 2, 3, 3, 4, 5, 6, 7, 8],
 'attendance': [50, 55, 60, 65, 70, 72, 80, 90, 95, 97],
 'result': ['Fail',
  'Fail',
  'Fail',
  'Fail',
  'Fail',
  'Pass',
  'Pass',
  'Pass',
  'Pass',
  'Pass']}
```

In [216…
```python
df=pd.DataFrame(data)
```

In [218…
```python
df
```

Out[218…

	Study_hours	attendance	result
0	1	50	Fail
1	2	55	Fail
2	2	60	Fail
3	3	65	Fail
4	3	70	Fail
5	4	72	Pass
6	5	80	Pass
7	6	90	Pass
8	7	95	Pass
9	8	97	Pass

Target Variable

In [22]:
```python
Y=df[['result']]
X=df.drop(labels=['result'],axis=1)
```

In [24]:
```python
Y
```

Out[24]:

	result
0	Fail
1	Fail
2	Fail
3	Fail
4	Fail
5	Pass
6	Pass
7	Pass
8	Pass
9	Pass

In [26]:
```python
X
```

Out[26]:

	Study_hours	attendance
0	1	50
1	2	55
2	2	60
3	3	65
4	3	70
5	4	72
6	5	80
7	6	90
8	7	95
9	8	97

Train-Test split

In [58]:
```python
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(X,Y,test_size=0.2,random_state=21)
```

Feature scaling

In [61]:
```python
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
```

In [63]:
```python
xtrain_scaled=sc.fit_transform(xtrain)
xtest_scaled=sc.fit_transform(xtest)
```

In [65]:
```python
xtrain_scaled
```

Out[65]:
```
array([[ 0.63193349,  0.83069211],
       [-0.74683049, -0.81424276],
       [-0.28724249, -0.353661  ],
       [ 0.1723455 ,  0.17271816],
       [-1.66600647, -1.80120368],
       [-0.74683049, -0.48525579],
       [ 1.09152148,  1.15967908],
       [ 1.55110947,  1.29127387]])
```

In [67]:
```python
xtest_scaled
```

Out[67]:
```
array([[ 0., -1.],
       [ 0.,  1.]])
```

Create Model

In [153…
```python
from sklearn.neighbors import KNeighborsClassifier
knn=KNeighborsClassifier(n_neighbors=3,weights='uniform',metric='euclidean',p=2)
```

In [155…
```python
model=knn.fit(xtrain_scaled,ytrain)
```

Make Prediction

In [158…
```python
y_pred=model.predict(xtest_scaled)
```

In [160…
```python
print(y_pred)
```

```
['Fail' 'Pass']
```

In [162…
```python
#ytest['y_pred']=y_pred
```

In [164…
```python
#ytest
```

Evalute the model

In [167…
```python
from sklearn.metrics import accuracy_score,classification_report

accuracy=accuracy_score(ytest,y_pred)
```

In [169…
```python
accuracy
```

Out[169…
```
0.5
```

In [171…
```python
classification_report(ytest,y_pred)
```

Out[171…
```
'              precision    recall  f1-score   support\n\n        Fail       1.00      0.50      0.67         2\n        Pass       0.00      0.00      0.00         0\n\n    accuracy
0.50         2\n   macro avg       0.50      0.25      0.33         2\nweighted avg       1.00      0.50      0.67         2\n'
```

In [173…
```python
from warnings import filterwarnings
filterwarnings('ignore')
```

In [175…
```python
study_hour=6
attendance=85

new_student=[[6,85]]

new_student_scaled=sc.fit_transform(new_student)
```

In [177…
```python
new_student_scaled
```

Out[177…
```
array([[0., 0.]])
```

In [179…
```python
prediction=model.predict(new_student_scaled)
print(prediction)
```

```
['Pass']
```

In [ ]:
```python
k=3
```

In [183…
```python
for k in range(1,7):
    knn=KNeighborsClassifier(n_neighbors=k)
    knn.fit(xtrain_scaled,ytrain)
    y_pred=knn.predict(xtest_scaled)
    accuracy=accuracy_score(ytest,y_pred)
    print(k,accuracy)
```

```
1 0.0
2 0.5
3 0.5
4 0.5
5 0.5
6 0.5
```

Standard scaler

In [ ]:
```
Age   25
Salary  30,000
```

In [ ]:
```
1000
800

80%
```

In [ ]:
```
Z-Score=

Z=x-mean/sd

x---->Original value
u--->mean
SD -->(Sigma)
Z--->Scaled value
```

In [186…
```python
Mean=50
SD=10
X=70

(X-Mean)/SD
```

Out[186…
```
2.0
```

In [ ]:
```
70
2

Negative --->Below average
0---->Around average
Positive  --->Above average
```

In [192…
```python
df=df[['Study_hours','attendance']]
```

In [194…
```python
df
```

Out[194…

	Study_hours	attendance
0	1	50
1	2	55
2	2	60
3	3	65
4	3	70
5	4	72
6	5	80
7	6	90
8	7	95
9	8	97

In [196…
```python
scaled_data=sc.fit_transform(df)
```

In [198…
```python
scaled_data
```

Out[198…
```
array([[-1.40186846, -1.4822006 ],
       [-0.94965283, -1.16549107],
       [-0.94965283, -0.84878154],
       [-0.49743719, -0.53207201],
       [-0.49743719, -0.21536248],
       [-0.04522156, -0.08867867],
       [ 0.40699407,  0.41805658],
       [ 0.8592097 ,  1.05147564],
       [ 1.31142533,  1.36818517],
       [ 1.76364096,  1.49486899]])
```

In [200…
```python
scaled_df=pd.DataFrame(
    scaled_data,columns=df.columns
)
```

In [202…
```python
scaled_df
```

Out[202…

	Study_hours	attendance
0	-1.401868	-1.482201
1	-0.949653	-1.165491
2	-0.949653	-0.848782
3	-0.497437	-0.532072
4	-0.497437	-0.215362
5	-0.045222	-0.088679
6	0.406994	0.418057
7	0.859210	1.051476
8	1.311425	1.368185
9	1.763641	1.494869

In [204…
```python
x=[1,2,3,4,5]
Z=x-mean/sd
n=5

1-3/4
```

In [ ]:
```
x  mean    var    sd     Z
1   3       1     1
2   3
3   3
4   3
5   3
```

Normalization

In [ ]:
```
Normalization transform values into a
specified range ,commanly 0-1
```

In [ ]:
```
MinMaxScaler=   X-Xmin/Xmax-Xmin

X--->Original value
Xmin --->Minimum value
Xmax  -->Max value
X' --->Normalized value

Min=10
Max=50
X=30
```

In [207…
```python
(30-10)/(50-10)
```

Out[207…
```
0.5
```

In [220…
```python
df=df[['Study_hours','attendance']]
```

In [222…
```python
df
```

Out[222…

	Study_hours	attendance
0	1	50
1	2	55
2	2	60
3	3	65
4	3	70
5	4	72
6	5	80
7	6	90
8	7	95
9	8	97

In [224…
```python
from sklearn.preprocessing import MinMaxScaler
ms=MinMaxScaler()

scaled_data=ms.fit_transform(df)
```

In [226…
```python
scaled_data
```

Out[226…
```
array([[0.        , 0.        ],
       [0.14285714, 0.10638298],
       [0.14285714, 0.21276596],
       [0.28571429, 0.31914894],
       [0.28571429, 0.42553191],
       [0.42857143, 0.46808511],
       [0.57142857, 0.63829787],
       [0.71428571, 0.85106383],
       [0.85714286, 0.95744681],
       [1.        , 1.        ]])
```

In [228…
```python
pd.DataFrame(scaled_data,columns=df.columns)
```

Out[228…

	Study_hours	attendance
0	0.000000	0.000000
1	0.142857	0.106383
2	0.142857	0.212766
3	0.285714	0.319149
4	0.285714	0.425532
5	0.428571	0.468085
6	0.571429	0.638298